

Lab 7: Decision Tree Classification on the Iris Dataset

Import Required Libraries

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

sns.set_theme(style="whitegrid")
pd.set_option("display.max_colwidth", None)
```

Task 1: Dataset Exploration

```
In [2]: # Load the Iris dataset as pandas objects
iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

# Combine features and readable target labels for exploration
iris_df = X.copy()
iris_df["target"] = y
iris_df["species"] = y.map(dict(enumerate(iris.target_names)))

print(f"Number of samples: {X.shape[0]}")
print(f"Number of features: {X.shape[1]}")
print("\nFeature names:")
for feature in iris.feature_names:
    print(f"- {feature}")
print("\nTarget classes:", list(iris.target_names))

print("\nFirst five records:")
display(iris_df.head())
```

Number of samples: 150

Number of features: 4

Feature names:

- sepal length (cm)
- sepal width (cm)
- petal length (cm)
- petal width (cm)

Target classes: [np.str_('setosa'), np.str_('versicolor'), np.str_('virginica')]

First five records:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target	species
0	5.1	3.5	1.4	0.2	0	setosa
1	4.9	3.0	1.4	0.2	0	setosa
2	4.7	3.2	1.3	0.2	0	setosa
3	4.6	3.1	1.5	0.2	0	setosa
4	5.0	3.6	1.4	0.2	0	setosa

```
In [3]: class_distribution = (
    iris_df["species"]
    .value_counts()
    .reindex(iris.target_names)
    .rename_axis("Class")
    .reset_index(name="Number of samples")
)
print("Class distribution:")
display(class_distribution)
```

```
plt.figure(figsize=(7, 4))
sns.countplot(data=iris_df, x="species", order=iris.target_names, color="steelblue")
plt.title("Iris Class Distribution")
plt.xlabel("Class")
plt.ylabel("Number of samples")
plt.tight_layout()
plt.show()
```

Class distribution:

	Class	Number of samples
0	setosa	50
1	versicolor	50
2	virginica	50



Task 2: Data Preparation

```
In [4]: X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training feature shape:", X_train.shape)
print("Testing feature shape:", X_test.shape)
print(f"Training samples: {len(X_train)} ({len(X_train) / len(X):.0%})")
print(f"Testing samples: {len(X_test)} ({len(X_test) / len(X):.0%})")
print("\nTraining class distribution:", y_train.value_counts().sort_index().to_dict())
print("Testing class distribution: ", y_test.value_counts().sort_index().to_dict())
```

Training feature shape: (120, 4)
 Testing feature shape: (30, 4)
 Training samples: 120 (80%)
 Testing samples: 30 (20%)

Training class distribution: {0: 40, 1: 40, 2: 40}
 Testing class distribution: {0: 10, 1: 10, 2: 10}

The dataset is divided so the model learns patterns from the training set, while the testing set evaluates its performance on unseen data. This provides a more realistic measure of generalization and helps detect overfitting.

Task 3: Building a Decision Tree Classifier

```
In [5]: # Train the default Decision Tree model
dt_default = DecisionTreeClassifier(random_state=42)
dt_default.fit(X_train, y_train)

# Predict the test labels
y_pred = dt_default.predict(X_test)
```

```

prediction_preview = pd.DataFrame({
    "Actual": [iris.target_names[i] for i in y_test.to_numpy()],
    "Predicted": [iris.target_names[i] for i in y_pred]
}, index=y_test.index).sort_index()

print("Predicted class labels:")
print(y_pred)
print("\nActual and predicted classes:")
display(prediction_preview)

```

Predicted class labels:

[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 1 2 2 1 0 2 0]

Actual and predicted classes:

	Actual	Predicted
7	setosa	setosa
10	setosa	setosa
14	setosa	setosa
18	setosa	setosa
20	setosa	setosa
22	setosa	setosa
28	setosa	setosa
38	setosa	setosa
42	setosa	setosa
49	setosa	setosa
51	versicolor	versicolor
56	versicolor	versicolor
57	versicolor	versicolor
58	versicolor	versicolor
63	versicolor	versicolor
69	versicolor	versicolor
75	versicolor	versicolor
77	versicolor	virginica
84	versicolor	versicolor
93	versicolor	versicolor
104	virginica	virginica
107	virginica	virginica
116	virginica	virginica
127	virginica	virginica
132	virginica	virginica
134	virginica	versicolor
138	virginica	virginica
140	virginica	virginica
141	virginica	virginica
147	virginica	virginica

```

In [6]: default_accuracy = accuracy_score(y_test, y_pred)
        default_cm = confusion_matrix(y_test, y_pred)

        print(f"Accuracy Score: {default_accuracy:.4f} ({default_accuracy:.2%})")
        print("\nConfusion Matrix:")
        print(default_cm)
        print("\nClassification Report:")

```

```

print(classification_report(y_test, y_pred, target_names=iris.target_names))

plt.figure(figsize=(6, 4))
sns.heatmap(
    default_cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=iris.target_names,
    yticklabels=iris.target_names
)
plt.title("Confusion Matrix: Default Decision Tree")
plt.xlabel("Predicted class")
plt.ylabel("Actual class")
plt.tight_layout()
plt.show()

```

Accuracy Score: 0.9333 (93.33%)

Confusion Matrix:

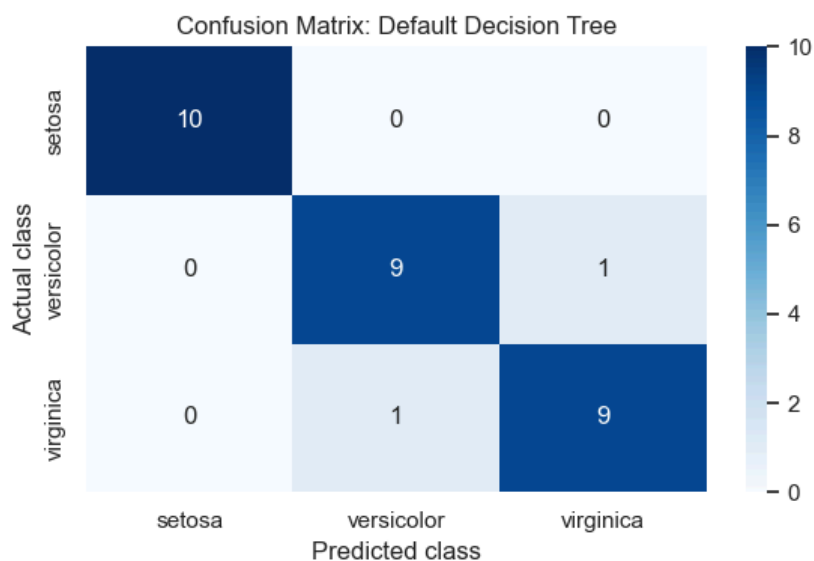
```

[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]

```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	0.90	0.90	0.90	10
virginica	0.90	0.90	0.90	10
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30



Task 3A: Stratified 5-Fold Cross-Validation

A single train-test split can be unusually easy or difficult, especially with a small dataset. Stratified 5-fold cross-validation evaluates the model five times while preserving the class proportions in every fold. Each sample is used for validation once and for training four times.

```

In [7]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

cv_models = {
    "Unrestricted tree": DecisionTreeClassifier(random_state=42),
    "Depth-3 tree": DecisionTreeClassifier(max_depth=3, random_state=42)
}

fold_rows = []
summary_rows = []

for model_name, model in cv_models.items():
    scores = cross_validate(
        model,

```

```

X,
y,
cv=cv,
scoring="accuracy",
return_train_score=True
)

for fold, (train_score, validation_score) in enumerate(
    zip(scores["train_score"], scores["test_score"]), start=1
):
    fold_rows.append({
        "Model": model_name,
        "Fold": fold,
        "Training Accuracy": train_score,
        "Validation Accuracy": validation_score
    })

mean_train = scores["train_score"].mean()
mean_validation = scores["test_score"].mean()
summary_rows.append({
    "Model": model_name,
    "Mean Training Accuracy": mean_train,
    "Mean Validation Accuracy": mean_validation,
    "Validation Std. Dev.": scores["test_score"].std(ddof=1),
    "Train-Validation Gap": mean_train - mean_validation
})

fold_results = pd.DataFrame(fold_rows)
cv_summary = pd.DataFrame(summary_rows)

print("Accuracy in each fold:")
display(
    fold_results.style.format({
        "Training Accuracy": "{:.2%}",
        "Validation Accuracy": "{:.2%}"
    })
)

print("Cross-validation summary:")
display(
    cv_summary.style.format({
        "Mean Training Accuracy": "{:.2%}",
        "Mean Validation Accuracy": "{:.2%}",
        "Validation Std. Dev.": "{:.2%}",
        "Train-Validation Gap": "{:.2%}"
    })
)

```

Accuracy in each fold:

	Model	Fold	Training Accuracy	Validation Accuracy
0	Unrestricted tree	1	100.00%	100.00%
1	Unrestricted tree	2	100.00%	96.67%
2	Unrestricted tree	3	100.00%	93.33%
3	Unrestricted tree	4	100.00%	96.67%
4	Unrestricted tree	5	100.00%	90.00%
5	Depth-3 tree	1	96.67%	96.67%
6	Depth-3 tree	2	97.50%	96.67%
7	Depth-3 tree	3	98.33%	93.33%
8	Depth-3 tree	4	96.67%	100.00%
9	Depth-3 tree	5	98.33%	93.33%

Cross-validation summary:

	Model	Mean Training Accuracy	Mean Validation Accuracy	Validation Std. Dev.	Train-Validation Gap
0	Unrestricted tree	100.00%	95.33%	3.80%	4.67%
1	Depth-3 tree	97.50%	96.00%	2.79%	1.50%

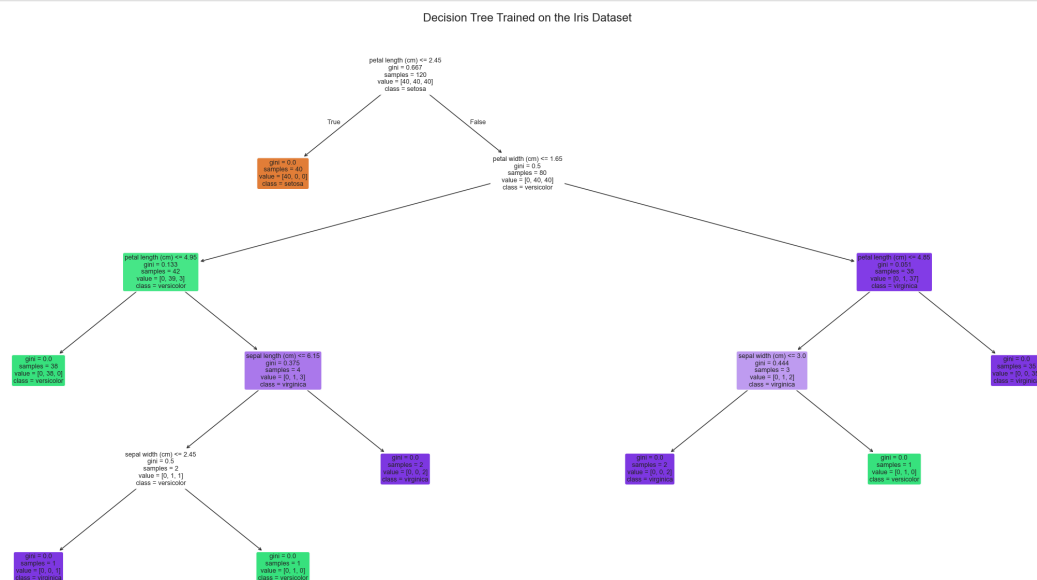
Cross-Validation Interpretation

- The **unrestricted tree** obtains **100.00% mean training accuracy** and **95.33% mean validation accuracy** with a standard deviation of **3.80%**. Its train-validation gap is **4.67 percentage points**.
- The **depth-3 tree** obtains **97.50% mean training accuracy** and **96.00% mean validation accuracy** with a standard deviation of **2.79%**. Its gap is only **1.50 percentage points**.

The original 100% holdout accuracy was therefore a favorable result rather than a guarantee of perfect performance on unseen data. The unrestricted tree shows **mild overfitting** because it memorizes every training fold but performs less accurately during validation. The depth-3 model has slightly higher and more stable validation performance with a much smaller gap, strengthening its recommendation.

Task 4: Visualizing the Decision Tree

```
In [8]: plt.figure(figsize=(24, 12))
plot_tree(
    dt_default,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True,
    rounded=True,
    fontsize=9
)
plt.title("Decision Tree Trained on the Iris Dataset", fontsize=16)
plt.tight_layout()
plt.show()
```



```
In [9]: tree = dt_default.tree_
root_feature_index = tree.feature[0]
root_feature = iris.feature_names[root_feature_index]
root_threshold = tree.threshold[0]

internal_node_ids = [
    node_id for node_id in range(tree.node_count)
    if tree.children_left[node_id] != tree.children_right[node_id]
]
leaf_node_ids = [
    node_id for node_id in range(tree.node_count)
    if tree.children_left[node_id] == tree.children_right[node_id]
]

print(f"Root node (node 0): {root_feature} <= {root_threshold:.2f}")
print(f"Total splitting/internal nodes (including root): {len(internal_node_ids)}")
print(f"Other internal nodes: {len(internal_node_ids) - 1}")
print(f"Internal node IDs: {internal_node_ids}")
print(f"Leaf nodes: {len(leaf_node_ids)}")
print(f"Leaf node IDs: {leaf_node_ids}")
print(f"Maximum depth: {dt_default.get_depth()}")
print(f"Feature selected for first split: {root_feature}")
```

Root node (node 0): petal length (cm) <= 2.45
Total splitting/internal nodes (including root): 7
Other internal nodes: 6
Internal node IDs: [0, 2, 3, 5, 6, 10, 11]
Leaf nodes: 8
Leaf node IDs: [1, 4, 7, 8, 9, 12, 13, 14]
Maximum depth: 5
Feature selected for first split: petal length (cm)

Interpretation of the Tree

- **Root node:** Node 0, which tests petal length (cm) <= 2.45.
- **Internal nodes:** There are 7 splitting nodes including the root, or 6 additional internal nodes below it. These nodes contain further feature tests.
- **Leaf nodes:** There are 8 terminal nodes. Each leaf gives the final predicted class and has no children.
- **Maximum depth: 5**, where the root is at depth 0.
- **First split feature: petal length (cm).**

It was chosen because this split produces a large reduction in Gini impurity by cleanly separating setosa, which has much shorter petals, from versicolor and virginica.

Task 5: Comparing Gini Index and Entropy

```
In [10]: criterion_models = {}
criterion_rows = []

for criterion in ["gini", "entropy"]:
    model = DecisionTreeClassifier(criterion=criterion, random_state=42)
    model.fit(X_train, y_train)
    criterion_models[criterion] = model
    criterion_rows.append({
        "Criterion": criterion.title(),
        "Testing Accuracy": accuracy_score(y_test, model.predict(X_test)),
        "Tree Depth": model.get_depth(),
        "Leaf Nodes": model.get_n_leaves(),
        "Root Feature": iris.feature_names[model.tree_.feature[0]]
    })

criterion_results = pd.DataFrame(criterion_rows)
criterion_results["Testing Accuracy"] = criterion_results["Testing Accuracy"].map(lambda x: f"{x:.2%}")
display(criterion_results)
```

	Criterion	Testing Accuracy	Tree Depth	Leaf Nodes	Root Feature
0	Gini	93.33%	5	8	petal length (cm)
1	Entropy	93.33%	5	8	petal length (cm)

Observation

Both criteria produce the same measured structure: **93.33% testing accuracy, depth 5, 8 leaves, and petal length as the root feature**. Therefore, **neither criterion produces a structurally simpler tree in this experiment**. Gini is usually cheaper to compute because it does not require logarithms, but that computational distinction does not change the depth or leaf count here.

Task 6: Effect of Maximum Tree Depth

```
In [11]: depth_observations = {
    1: "Underfits; the tree is too shallow to separate all three classes.",
    2: "Compact model with high, but not perfect, testing accuracy.",
    3: "Best testing accuracy and a good complexity-performance balance.",
    4: "More complex than depth 3, but testing accuracy decreases.",
    None: "Perfect training accuracy but lower testing accuracy; strongest overfitting evidence."
}

depth_rows = []
depth_models = {}
for max_depth in [1, 2, 3, 4, None]:
    model = DecisionTreeClassifier(max_depth=max_depth, random_state=42)
    model.fit(X_train, y_train)
    depth_models[max_depth] = model
```

```

depth_rows.append({
    "Max Depth": "None" if max_depth is None else str(max_depth),
    "Training Accuracy": model.score(X_train, y_train),
    "Testing Accuracy": model.score(X_test, y_test),
    "Tree Depth": model.get_depth(),
    "Observation": depth_observations[max_depth]
})

depth_results_numeric = pd.DataFrame(depth_rows)
depth_results = depth_results_numeric.copy()
for column in ["Training Accuracy", "Testing Accuracy"]:
    depth_results[column] = depth_results[column].map(lambda x: f"{x:.2%}")
display(depth_results)

```

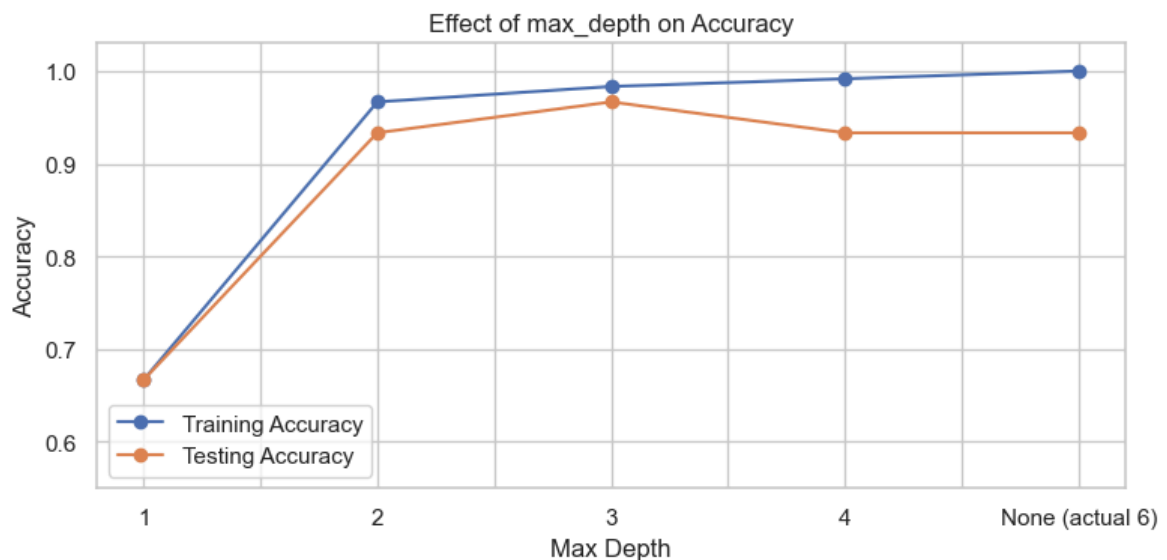
	Max Depth	Training Accuracy	Testing Accuracy	Tree Depth	Observation
0	1	66.67%	66.67%	1	Underfits; the tree is too shallow to separate all three classes.
1	2	96.67%	93.33%	2	Compact model with high, but not perfect, testing accuracy.
2	3	98.33%	96.67%	3	Best testing accuracy and a good complexity-performance balance.
3	4	99.17%	93.33%	4	More complex than depth 3, but testing accuracy decreases.
4	None	100.00%	93.33%	5	Perfect training accuracy but lower testing accuracy; strongest overfitting evidence.

```

In [12]: depth_plot = depth_results_numeric.copy()
depth_plot["Max Depth"] = ["1", "2", "3", "4", "None (actual 6)"]

ax = depth_plot.plot(
    x="Max Depth",
    y=["Training Accuracy", "Testing Accuracy"],
    marker="o",
    figsize=(8, 4),
    ylim=(0.55, 1.03)
)
ax.set_title("Effect of max_depth on Accuracy")
ax.set_ylabel("Accuracy")
plt.tight_layout()
plt.show()

```



Answers

- **Underfitting model:** `max_depth=1` . Its training and testing accuracies are only **66.67%**, showing that it cannot capture enough structure.
- **Best generalization:** `max_depth=3` . It gives the highest testing accuracy, **96.67%**, with **98.33% training accuracy**, depth 3, and 5 leaves.
- **Most likely to overfit:** `max_depth=None` . It grows to depth 5 and reaches **100% training accuracy**, but testing accuracy falls to **93.33%**. The larger train-test gap and extra complexity provide evidence of overfitting on this split.

Task 7: Effect of `min_samples_split`

```
In [13]: split_observations = {
    2: "Default; deepest and most detailed tree.",
    5: "Simpler tree and the best testing accuracy in this experiment.",
    10: "Same compact structure and testing accuracy as 5.",
    20: "Same compact structure and testing accuracy as 5 and 10."
}

split_rows = []
for min_split in [2, 5, 10, 20]:
    model = DecisionTreeClassifier(min_samples_split=min_split, random_state=42)
    model.fit(X_train, y_train)
    split_rows.append({
        "min_samples_split": min_split,
        "Accuracy": model.score(X_test, y_test),
        "Tree Depth": model.get_depth(),
        "Number of Leaf Nodes": model.get_n_leaves(),
        "Observation": split_observations[min_split]
    })

split_results = pd.DataFrame(split_rows)
split_results["Accuracy"] = split_results["Accuracy"].map(lambda x: f"{x:.2%}")
display(split_results)
```

	min_samples_split	Accuracy	Tree Depth	Number of Leaf Nodes	Observation
0	2	93.33%	5	8	Default; deepest and most detailed tree.
1	5	96.67%	3	5	Simpler tree and the best testing accuracy in this experiment.
2	10	96.67%	3	5	Same compact structure and testing accuracy as 5.
3	20	96.67%	3	5	Same compact structure and testing accuracy as 5 and 10.

Increasing `min_samples_split` requires more samples before a node may be divided. This blocks splits in small nodes and therefore generally reduces depth and the number of leaves. In this experiment, increasing it from 2 to 5 reduces depth from **5 to 3** and leaves from **8 to 5**, while testing accuracy improves from **93.33% to 96.67%**. Values 5, 10, and 20 happen to produce the same tree structure.

Task 8: Effect of `min_samples_leaf`

```
In [14]: leaf_observations = {
    1: "Default; allows very small leaves and the most detailed tree.",
    2: "Lower complexity with unchanged testing accuracy.",
    5: "Stronger regularization; compact tree with unchanged testing accuracy.",
    10: "Same structure and testing accuracy as 5 on this split."
}

leaf_rows = []
for min_leaf in [1, 2, 5, 10]:
    model = DecisionTreeClassifier(min_samples_leaf=min_leaf, random_state=42)
    model.fit(X_train, y_train)
    leaf_rows.append({
        "min_samples_leaf": min_leaf,
        "Accuracy": model.score(X_test, y_test),
        "Tree Depth": model.get_depth(),
        "Number of Leaf Nodes": model.get_n_leaves(),
        "Observation": leaf_observations[min_leaf]
    })

leaf_results = pd.DataFrame(leaf_rows)
leaf_results["Accuracy"] = leaf_results["Accuracy"].map(lambda x: f"{x:.2%}")
display(leaf_results)
```

	min_samples_leaf	Accuracy	Tree Depth	Number of Leaf Nodes	Observation
0	1	93.33%	5	8	Default; allows very small leaves and the most detailed tree.
1	2	93.33%	4	6	Lower complexity with unchanged testing accuracy.
2	5	93.33%	3	5	Stronger regularization; compact tree with unchanged testing accuracy.
3	10	93.33%	3	5	Same structure and testing accuracy as 5 on this split.

A larger `min_samples_leaf` forces every terminal node to represent more training samples. This prevents tiny leaves that may memorize isolated or noisy records, reducing variance and overfitting risk. If set too high, it can oversmooth real class boundaries and cause underfitting. Here, raising the value from 1 to 5 reduces depth from **5 to 3** and leaves from **8 to 5**, while testing accuracy remains **93.33%**. Values 5 and 10 produce the same structure on this split.

Task 9: Final Questions and Recommendations

1. What is the role of the criterion parameter?

criterion defines how the quality of a candidate split is measured. Gini impurity and entropy both measure class mixing; the algorithm chooses the split with the greatest weighted impurity reduction. Gini uses squared class probabilities, while entropy uses their logarithms.

2. How does max_depth influence underfitting and overfitting?

A very small max_depth limits the rules so strongly that the tree may underfit, as demonstrated by depth 1. A large or unlimited depth increases flexibility and can fit fine details or noise, increasing overfitting risk. An intermediate depth can retain the useful class boundaries while limiting variance.

3. Why do larger min_samples_split and min_samples_leaf values produce simpler trees?

min_samples_split prevents nodes with too few records from splitting, while min_samples_leaf prevents splits that would create very small children. Both rules stop growth in the lower parts of the tree, resulting in fewer splits, shallower paths, and fewer leaves.

4. Which hyperparameter had the greatest impact?

max_depth had the greatest observed impact on performance. Across its tested range, testing accuracy changed from 66.67% to 96.67%, training accuracy changed from 66.67% to 100%, and actual depth changed from 1 to 5. In comparison, changing min_samples_split changed testing accuracy only from 93.33% to 96.67%, while min_samples_leaf left testing accuracy at 93.33% for every tested value. Changing Gini to entropy made no difference to the reported metrics.

5. Recommended model

The most suitable model from these experiments is:

```
DecisionTreeClassifier(criterion="gini", max_depth=3, random_state=42)
```

It is recommended because it achieves the highest stratified-holdout testing accuracy, **96.67%**, with **98.33% training accuracy**, depth 3, and only 5 leaves. Its 5-fold cross-validation accuracy is also **96.00%**, with a smaller train-validation gap than the unrestricted tree. It therefore performs better and is substantially simpler than the default depth-5, 8-leaf tree. Gini and entropy perform identically here, so Gini is retained as the standard and computationally simpler default.